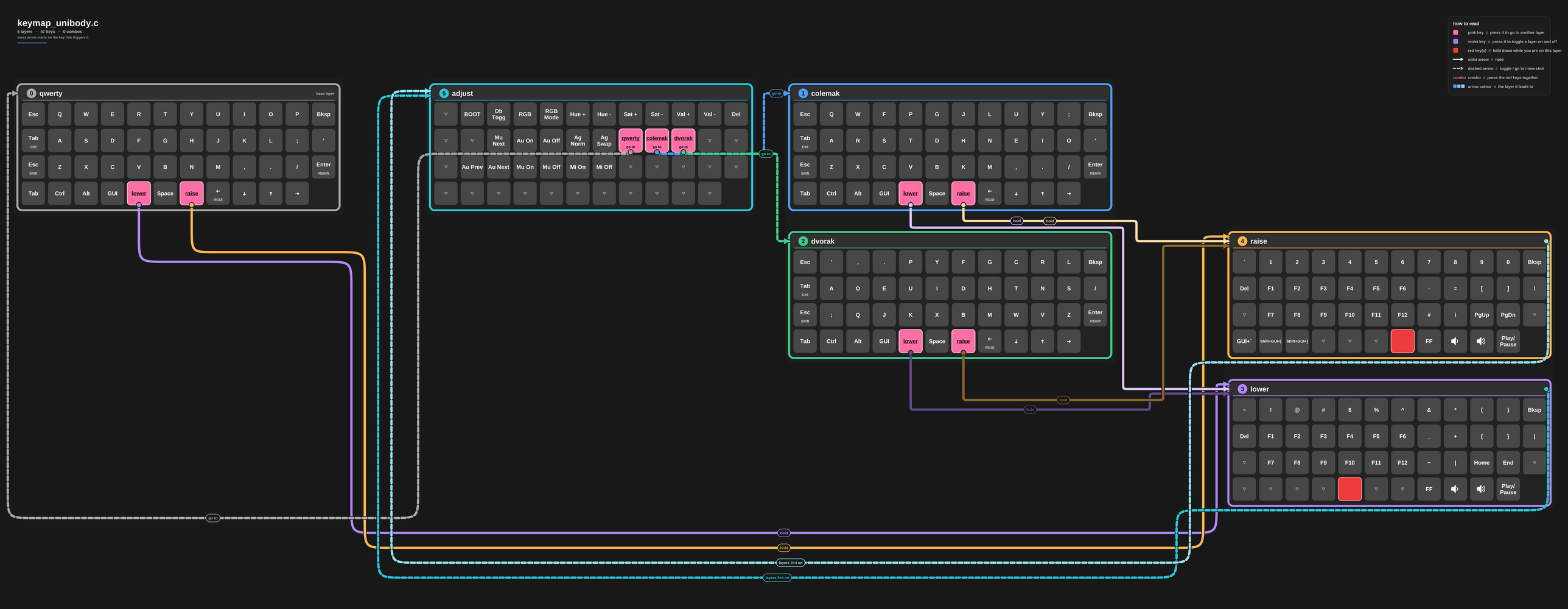
keymap_unibody.c

6 layers · 47 keys · 0 combos

every arrow starts on the key that triggers it



keymap_unibody.c

keymap source, comments removed

```
#include QMK_KEYBOARD_H

enum planck_layers {
  _QWERTY,
  _COLEMAK,
  _DVORAK,
  _LOWER,
  _RAISE,
  _ADJUST
};

enum planck_keycodes {
  QWERTY = SAFE_RANGE,
  COLEMAK,
  DVORAK,
  LOWER,
  RAISE
};

const uint16_t PROGMEM keymaps[][MATRIX_ROWS][MATRIX_COLS] = {

  [_QWERTY] = LAYOUT_planck_mit(
    KC_ESC,               KC_Q,    KC_W,    KC_E,    KC_R,    KC_T,    KC_Y,    KC_U,    KC_I,                   KC_O,    KC_P,    KC_BSPC,
    MT(MOD_LCTL, KC_TAB), KC_A,    KC_S,    KC_D,    KC_F,    KC_G,    KC_H,    KC_J,    KC_K,                   KC_L,    KC_SCLN, KC_QUOT,
    MT(MOD_LSFT, KC_ESC), KC_Z,    KC_X,    KC_C,    KC_V,    KC_B,    KC_N,    KC_M,    KC_COMM,                KC_DOT,  KC_SLSH, MT(MOD_RSFT, KC_ENT),
    KC_TAB,               KC_LCTL, KC_LALT, KC_LGUI, LOWER,       KC_SPC,       RAISE,   MT(MOD_RGUI, KC_LEFT), KC_DOWN, KC_UP,   KC_RGHT
  ),

  [_COLEMAK] = LAYOUT_planck_mit(
    KC_ESC,               KC_Q,    KC_W,    KC_F,    KC_P,    KC_G,    KC_J,    KC_L,    KC_U,                   KC_Y,    KC_SCLN, KC_BSPC,
    MT(MOD_LCTL, KC_TAB), KC_A,    KC_R,    KC_S,    KC_T,    KC_D,    KC_H,    KC_N,    KC_E,                   KC_I,    KC_O,    KC_QUOT,
    MT(MOD_LSFT, KC_ESC), KC_Z,    KC_X,    KC_C,    KC_V,    KC_B,    KC_K,    KC_M,    KC_COMM,                KC_DOT,  KC_SLSH, MT(MOD_RSFT, KC_ENT),
    KC_TAB,               KC_LCTL, KC_LALT, KC_LGUI, LOWER,       KC_SPC,       RAISE,   MT(MOD_RGUI, KC_LEFT), KC_DOWN, KC_UP,   KC_RGHT
  ),

  [_DVORAK] = LAYOUT_planck_mit(
    KC_ESC,               KC_QUOT, KC_COMM, KC_DOT,  KC_P,  KC_Y,    KC_F,    KC_G,    KC_C,                   KC_R,    KC_L,    KC_BSPC,
    MT(MOD_LCTL, KC_TAB), KC_A,    KC_O,    KC_E,    KC_U,  KC_I,    KC_D,    KC_H,    KC_T,                   KC_N,    KC_S,    KC_SLSH,
    MT(MOD_LSFT, KC_ESC), KC_SCLN, KC_Q,    KC_J,    KC_K,  KC_X,    KC_B,    KC_M,    KC_W,                   KC_V,    KC_Z,    MT(MOD_RSFT, KC_ENT),
    KC_TAB,               KC_LCTL, KC_LALT, KC_LGUI, LOWER,     KC_SPC,       RAISE,   MT(MOD_RGUI, KC_LEFT), KC_DOWN, KC_UP,   KC_RGHT
  ),

  [_LOWER] = LAYOUT_planck_mit(
    KC_TILD, KC_EXLM, KC_AT,   KC_HASH, KC_DLR,  KC_PERC, KC_CIRC, KC_AMPR,    KC_ASTR,    KC_LPRN, KC_RPRN, KC_BSPC,
    KC_DEL,  KC_F1,   KC_F2,   KC_F3,   KC_F4,   KC_F5,   KC_F6,   KC_UNDS,    KC_PLUS,    KC_LCBR, KC_RCBR, KC_PIPE,
    _______, KC_F7,   KC_F8,   KC_F9,   KC_F10,  KC_F11,  KC_F12,  S(KC_NUHS), S(KC_NUBS), KC_HOME, KC_END,  _______,
    _______, _______, _______, _______, _______,     _______,      _______,    KC_MFFD,    KC_VOLD, KC_VOLU, KC_MPLY
  ),

  [_RAISE] = LAYOUT_planck_mit(
    KC_GRV,         KC_1,                   KC_2,                   KC_3,    KC_4,    KC_5,    KC_6,    KC_7,    KC_8,    KC_9,    KC_0,    KC_BSPC,
    KC_DEL,         KC_F1,                  KC_F2,                  KC_F3,   KC_F4,   KC_F5,   KC_F6,   KC_MINS, KC_EQL,  KC_LBRC, KC_RBRC, KC_BSLS,
    _______,        KC_F7,                  KC_F8,                  KC_F9,   KC_F10,  KC_F11,  KC_F12,  KC_NUHS, KC_NUBS, KC_PGUP, KC_PGDN, _______,
    LGUI(KC_GRAVE), LGUI(LSFT(KC_LBRC)),    LGUI(LSFT(KC_RBRC)),    _______, _______, _______,          _______, KC_MFFD, KC_VOLD, KC_VOLU, KC_MPLY
  ),

  [_ADJUST] = LAYOUT_planck_mit(
    _______, QK_BOOT, DB_TOGG, RGB_TOG, RGB_MOD, RGB_HUI, RGB_HUD, RGB_SAI, RGB_SAD,  RGB_VAI, RGB_VAD, KC_DEL ,
    _______, _______, MU_NEXT, AU_ON,   AU_OFF,  AG_NORM, AG_SWAP, QWERTY,  COLEMAK,  DVORAK,  _______, _______,
    _______, AU_PREV, AU_NEXT, MU_ON,   MU_OFF,  MI_ON,   MI_OFF,  _______, _______,  _______, _______, _______,
    _______, _______, _______, _______, _______,     _______,      _______, _______,  _______, _______, _______
  )

};

bool process_record_user(uint16_t keycode, keyrecord_t *record) {
  switch (keycode) {
    case QWERTY:
      if (record->event.pressed) {
        print("mode just switched to qwerty and this is a huge string\n");
        set_single_persistent_default_layer(_QWERTY);
      }
      return false;
      break;
    case COLEMAK:
      if (record->event.pressed) {
        set_single_persistent_default_layer(_COLEMAK);
      }
      return false;
      break;
    case DVORAK:
      if (record->event.pressed) {
        set_single_persistent_default_layer(_DVORAK);
      }
      return false;
      break;
    case LOWER:
      if (record->event.pressed) {
        layer_on(_LOWER);
        update_tri_layer(_LOWER, _RAISE, _ADJUST);
      } else {
        layer_off(_LOWER);
        update_tri_layer(_LOWER, _RAISE, _ADJUST);
      }
      return false;
      break;
    case RAISE:
      if (record->event.pressed) {
        layer_on(_RAISE);
        update_tri_layer(_LOWER, _RAISE, _ADJUST);
      } else {
        layer_off(_RAISE);
        update_tri_layer(_LOWER, _RAISE, _ADJUST);
      }
      return false;
      break;
  }
  return true;
}
```